编译器设计专题实验课2025

2025.05

一、实验七

- 实验七内容

实验七设计存储格式、构建内存映像
实验八产生调用活动记录的代码序列产生可执行代码

一、实验回顾

- 实验分c/c++、python实现

Python有大量的库可以调用，实际上需要完成的代码量比C/C++要少很多

实验2的python部分：re这个库import以后，此处内容等同于flex.l **的内容，因此实验2的选做部分代码是让同学如果有时间精力可以去思考另一种实现方式**

```
Token stream for .\1.src:
<INT, int>
<ID, main>
<LPAR, (>
<RPAR, )>
<LBR, {>
<INT, int>
<ID, x>
<ASG, =>
<INT_NUM, 5>
<SEMI, ;>
```

窗口(W) 帮助(H) conpilercode - Scan.py

SA5.py | Scan.py

```
77      return KEYWORDS.get(s, TokenType.ID)
78
79  # Token patterns (order matters!)
80  TOKEN_PATTERNS = [
81      # Whitespace (skip)
82      (TokenType.END, r"\s+"),
83
84      # Comments (skip)
85      (TokenType.END, r"//.*"),
86      (TokenType.END, r"/\*.*?\*/"),
87
88      # Keywords (must come before ID)
89      (TokenType.INT, r"int\b"),
90      (TokenType.FLOAT, r"float\b"),
91      (TokenType.VOID, r"void\b"),
92      (TokenType.IF, r"if\b"),
93      (TokenType.ELSE, r"else\b"),
94      (TokenType.WHILE, r"while\b"),
95      (TokenType.RETURN, r"return\b"),
96      (TokenType.PRINT, r"print\b"),
97
98      # Numbers
99      (TokenType.FLOAT_NUM, r"-?\d+\.\d+"),
100     (TokenType.FLOAT_NUM, r"-?\d+\."),      # Handl
101     (TokenType.FLOAT_NUM, r"-?\.\d+"),      # Handl
102     (TokenType.INT_NUM, r"-?\d+"),
103
104     # Identifiers
105     (TokenType.ID, r"[a-zA-Z_][a-zA-Z0-9_]*"),
106
107     # Operators
108     (TokenType.REL_OP, r"==|!=|<=|>=|<|>"),
109     (TokenType.LPAR, r"\("),
110     (TokenType.RPAR, r"\)"),
111     (TokenType.LBR, r"\{"),
112     (TokenType.RBR, r"\}"),
113     (TokenType.LBRACK, r"\["),
114     (TokenType.RBRACK, r"\]"),
115     (TokenType.SEMI, r";"),
116     (TokenType.COMMA, r","),
```

一、实验回顾

- 实验分c/c++、python实现

Flex能做的事情：使用正则式来匹配词法

{}里面的内容是操作代码，我们可以设置让token打印处理。

Flex **(文件名).l

此时当前目录下多了一个“lex.yy.c” 文件，把这个 C 文件编译并运行一遍。

$ gcc -o scanner lex.yy.c

生成可执行程序，scanner，运行sacnner

$ ./scanner

然后在终端敲入代码(或输入文件)，则可识别词法

```
%%

[ \t\n]+   ;

"+"      { add_token(ADD, yytext, ""); }
"-"      { add_token(SUB, yytext, ""); }
"*"      { add_token(MUL, yytext, ""); }
"="      { add_token(ASG, yytext, ""); }
"=="     { add_token(ROP, yytext, "=="); }
"<="     { add_token(ROP, yytext, "<="); }
"<"      { add_token(ROP, yytext, "<"); }
"("      { add_token(LPA, yytext, ""); }
")"      { add_token(RPA, yytext, ""); }
"["      { add_token(LBK, yytext, ""); }
"]"      { add_token(RBK, yytext, ""); }
"{"      { add_token(LBR, yytext, ""); }
"}"      { add_token(RBR, yytext, ""); }
","      { add_token(CMA, yytext, ""); }
":"      { add_token(SCO, yytext, ""); }
";"      { add_token(SEM, yytext, ""); }

"int"    { add_token(INT_KW, yytext, ""); }
"float"  { add_token(FLOAT_KW, yytext, ""); }
"if"     { add_token(IF, yytext, ""); }
"else"   { add_token(ELSE, yytext, ""); }
"while"  { add_token(WHILE, yytext, ""); }
"return" { add_token(RETURN, yytext, ""); }
"input"  { add_token(INPUT, yytext, ""); }
"print"  { add_token(PRINT, yytext, ""); }
"void"   { add_token(VOID, yytext, ""); }

{ID}     { add_token(ID, yytext, yytext); }
```

一、实验回顾

- 实验分c/c++、python实现

对比flex和python关于词法部分的内容，flex%%和%%中间是正则是定义。

关键字部分，非常相似，

```
"int"    { add_token(INT_KW, yytext, ""); }
"float"  { add_token(FLOAT_KW, yytext, ""); }
"if"     { add_token(IF, yytext, ""); }
"else"   { add_token(ELSE, yytext, ""); }
"while"  { add_token(WHILE, yytext, ""); }
"return" { add_token(RETURN, yytext, ""); }
"input"  { add_token(INPUT, yytext, ""); }
"print"  { add_token(PRINT, yytext, ""); }
"void"   { add_token(VOID, yytext, ""); }
```

```
# Keywords (must come before ID)
(TokenType.INT, r"int\b"),
(TokenType.FLOAT, r"float\b"),
(TokenType.VOID, r"void\b"),
(TokenType.IF, r"if\b"),
(TokenType.ELSE, r"else\b"),
(TokenType.WHILE, r"while\b"),
(TokenType.RETURN, r"return\b"),
(TokenType.PRINT, r"print\b"),
```

纯C代码 scanner程序
头文件和c程序400多行

```
static Token read_token_int(void) {
    int c = getc_nonspace();

    switch (c) {
        case '0' ... '9':
            return read_number(c);
        case 'a' ... 'z':
        case 'A' ... 'Z':
        case '_':
            return read_ident(c);
        case '/': {
            int next_c = getc(stdin);
            if (next_c == '/') {
                skip_line_comment();
                return read_token_int();
            }
```

一、实验回顾

- 实验2如果是c++实现scanner，200-300行左右
- 需要大量判断和转化
- 如果使用DFA实现，代码量更大，出错部分较少，且可以用makefile打包成整个项目
- 大部分选做工作量不小，可能涉及另外的知识体系，而必做部分的内容完成相当好，报告分数也很乐观。

```
41  // 初始化词法分析器
42  void init() {
43      sourceCodeIndex = 0;
44      currentChar = nextchart();
45      lastKind = -1;
46      lastLoc = -1;
47      buffer.clear();
48  }
49
50  // 清理词法分析器
51  void cleanup() {
52      buffer.clear();
53  }
54
55  // 判断字符是否为字母
56  bool isLetter(char c) {
57      return (c >= 'a' && c <= 'z'
58  }
59
60  // 判断字符是否为数字
61  bool isDigit(char c) {
62      return c >= '0' && c <= '9';
63  }
64
```

一、实验回顾

```
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# gcc -S scanner.cpp
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# ls
```

- 执行过程
- gcc -S（大写S）可生成汇编文件
- 使用cat打开
- 当前是X86的汇编代码，gcc默认
- 和当前平台的汇编代码一致
- 最终可执行

root@dsw-544537-65f46cbf ×

```
testb   %al, %al
jne     .L121
movl    -120(%rbp), %eax
movslq  %eax, %rdx
movq    -104(%rbp), %rax
addq    %rdx, %rax
movb    $0, (%rax)
leaq    .LC31(%rip), %rax
movq    %rax, %rsi
leaq    out(%rip), %rax
movq    %rax, %rdi
call    _ZStlsISt11char_traitsIcEERSt13basic_
movq    %rax, %rdx
movq    -104(%rbp), %rax
movq    %rax, %rsi
movq    %rdx, %rdi
call    _ZStlsISt11char_traitsIcEERSt13basic_
movq    %rax, %rdx
leaq    .LC24(%rip), %rax
movq    %rax, %rsi
movq    %rdx, %rdi
call    _ZStlsISt11char_traitsIcEERSt13basic_
movq    _ZSt4endlIcSt11char_traitsIcEERSt13ba
```

一、实验回顾

- 执行过程
- X86的架构下，从.cpp 文件到.s文件到.o文件到可执行程序

```
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace#
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# gcc -S -o scanner.s scanner.cpp
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# as -o scanner.o scanner.s
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# g++ -o scanner scanner.o
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace# ls scanner
scanner
root@dsw-544537-65f46cbf78-t8scb:/mnt/workspace#
```

- 以实验二为例表示实验八的结果是可执行代码，中间代码通过编译，模拟器（spim）等过程可生成可执行代码，因此验收可以是三地址、三元式、四元式，三地址代码或者.s 代码或者可执行程序

一、实验回顾

- 关于验收代码库，感谢同学提示，经过修正目前可用。

```
selected_file = random.choice(src_files)
input_file = os.path.join(codebase_dir, selected_file)
print(f"\nSelected file for analysis: {input_file}")
```

- 随机选择一个整数，拼接成文件名

```
#文件名等于seed选出随机数种子+后缀src

selected_file = f"{random.randint( a: 1,  b: 33)}.src"
print(selected_file)
```

```
17.src

Selected file for analysis: 代码库\17.src
```

- 输出语义分析结果

```
State: 4, Token: ID, Action: s10
State: 10, Token: LPAR, Action: s11
State: 11, Token: INT, Action: s6
State: 6, Token: ID, Action: r8
State: 15, Token: ID, Action: s28
State: 28, Token: RPAR, Action: r15
State: 17, Token: RPAR, Action: r13
State: 16, Token: RPAR, Action: s29
State: 29, Token: LBR, Action: s42
```

一、实验回顾

- 文法的解析

以前ppt解析了词法，未定义的部分在不引入冲突的情况下可以自行增加

P → Ď Š

Ď → ε | Ď D ;

D → T d | T d[i] | T d (Ă) {Ď Š}

T → int | float | void

Ă → ε | Ă A ;

A → T d | T d[] | T d(T)

Š → S | Š ; S

S → d = E | d[E] = E | if (B) S | if (B) S else S | while (B) S | return E | {Š} | d(Ř) |

E → i | f | d | d[E] | E + E | E * E | (E) | d(Ř)

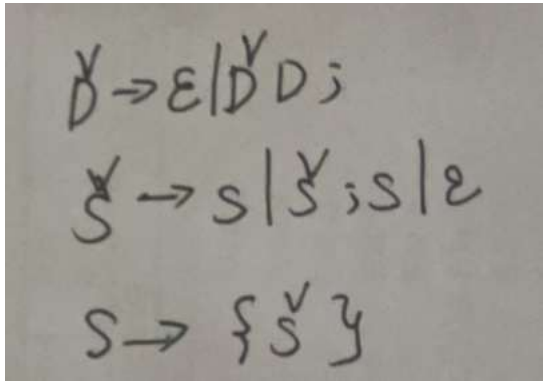
B →E r E | E

Ř → ε | Ř R ,

R → E | d[] | d()

Print/input可作为终结符

- S → d = E | d[E] = E | if (B) S | if (B) S else S | while (B) S | return E | {Š} | d(Ř) | input E|print E
- S 如果可以→ ε，c的主函数形式可以支持
- 关于文法的一些冲突



一、实验回顾

- 参考同学提供的内容，进行修正，大家可以自行修改或者重定义

```
Prog -> DeclList | StmtList
DeclList -> DeclList Decl | ε  # 或者没有声明，或者列表
Decl -> Type ID |   Type ID LBRACK INT_NUM RBRACK | FunDecl  |  #D → T d | T d[i] | T d (Ă) {Ď Š}
Type -> INT | FLOAT | VOID # 三种类型
ParamList -> ParamList Param  SEMI | Param | ε # Ă → ε | Ă A ;
Param -> Type ID | Type ID LBRACK RBRACK   | Type ID LPAR Type  RPAR
StmtList -> StmtList  SEMI Stmt | SEMI | ε
#CompStmt -> LBR StmtList RBR  # {Š}
Stmt -> VarDecl | OtherStmt  # S →  d(Ř)
OtherStmt -> ExprStmt | CompStmt | IfStmt | LoopStmt | RetStmt  #  if (B) S | if (B) S else S | while (B) S | return E
IfStmt -> IF LPAR BE RPAR Stmt  | IF LPAR BE RPAR Stmt ELSE Stmt
LoopStmt -> WHILE LPAR  BE RPAR Stmt
RetStmt -> RETURN ExprStmt
Stmt -> ID ASG Expr | ID LBRACK Expr RBRACK ASG Expr |  ID LPAR ArgList RPAR  | SimpExpr
##S → d = E | d[E] = E | d(Ř)
BE -> Expr RE Expr | ε
ArgList ->  ε | ArgList  Arg COMMA
Arg -> Expr  | ID LPAR RBAR   | ID LPBAR  RPBAR
AddExpr -> AddExpr ADD Term | Term ##此次定义了优先级避免+ * 冲突
Term -> Term MUL Fact | Fact
Fact -> ID | ID LBRACK Expr RBRACK | INT_NUM | FLOAT_NUM | LPAR Expr RPAR #  i | f | d | d[E] || (E) |
```

一、实验回顾

- 关于实验四，使用makefile构造工程，make和make clean

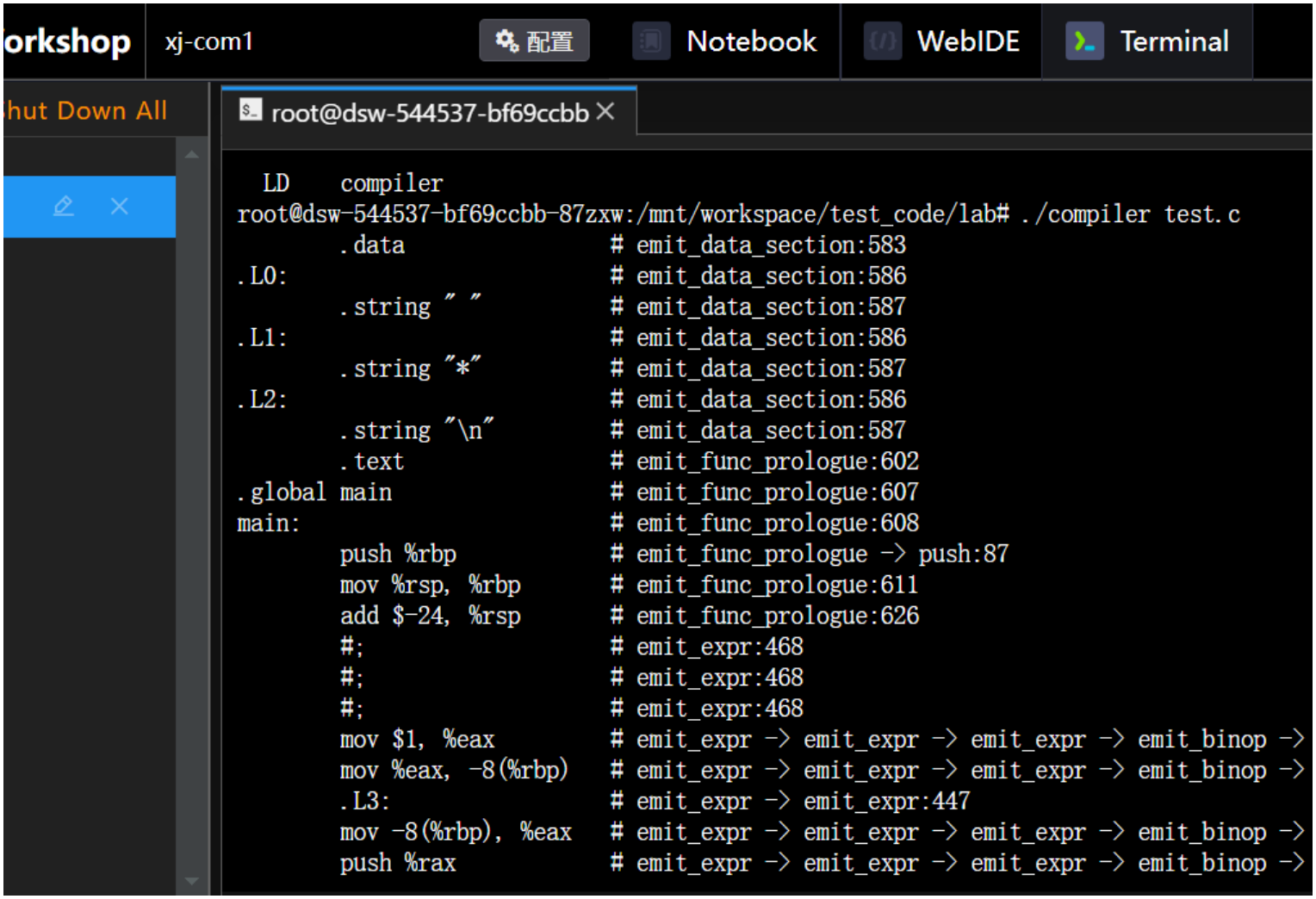
```
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab# make clean
rm -f compiler  lexer.o codegen.o parser.o verbose.o main.o .lexer.o.d .codeg
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab# make
  CC    lexer.o
  CC    codegen.o
  CC    parser.o
  CC    verbose.o
  CC    main.o
  LD    compiler
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab#
```

- 选择PAI-DSW环境：notebook，可以跑py，ipynb程序，可以运行c/c++，可以模拟linux环境，相对来说减少安装和环境配置，较为友好




```
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab# ./compiler
compiler [options] filename
OPTIONS
  -o filename           输出到指定文件
  --dump-ast            打印抽象语法树(AST)
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab# ./compiler  ../../1.src
parser.c:257: ';' expected, but got }
root@dsw-544537-759bc9dd8b-j8nkf:/mnt/workspace/test_code/lab#
```

一、实验回顾


orkshop
xj-com1
配置
Notebook
WebIDE
Terminal
hut Down All
root@dsw-544537-bf69ccbb
LD compiler
root@dsw-544537-bf69ccbb-87zxw:/mnt/workspace/test_code/lab# ./compiler test.c
.data # emit_data_section:583
.L0: # emit_data_section:586
.string " " # emit_data_section:587
.L1: # emit_data_section:586
.string "*" # emit_data_section:587
.L2: # emit_data_section:586
.string "\n" # emit_data_section:587
.text # emit_func_prologue:602
.global main # emit_func_prologue:607
main: # emit_func_prologue:608
push %rbp # emit_func_prologue -> push:87
mov %rsp, %rbp # emit_func_prologue:611
add $-24, %rsp # emit_func_prologue:626
#; # emit_expr:468
#; # emit_expr:468
#; # emit_expr:468
mov $1, %eax # emit_expr -> emit_expr -> emit_expr -> emit_binop ->
mov %eax, -8(%rbp) # emit_expr -> emit_expr -> emit_expr -> emit_binop ->
.L3: # emit_expr -> emit_expr:447
mov -8(%rbp), %eax # emit_expr -> emit_expr -> emit_expr -> emit_binop ->
push %rax # emit_expr -> emit_expr -> emit_expr -> emit_binop ->

二、实验七、八内容

- 实验七根据实验六的符号表内容，可输出内存地址和偏移等信息；实验八则根据中间代码信息，可输出目标代码、可执行代码等，本实验输出如mips、X86，arm不同指令集的汇编程序。
- 按教学进度，7，8内容合并，输入可以是三地址代码，三、四元式等中间代码，也可以是源代码，输出为汇编.s程序
- 本次实验没有作业，一共6次作业，验收程序可以实现到中间代码部分，也可以实现到汇编程序部分。
- 成绩构成：实验报告（6次，70%）+程序验收+（平时考勤10%）
- 验收内容：选中一个测试代码，看实现结果。如果程序设计的文法不是实验2的文法，逻辑部分重合，也认为通过
- 如某次实验报告逾期，当次报告成绩不会不及格
- 成绩发布之前，会在思源学堂上传实验报告成绩。

二、实验七、八内容

- 按标准要求每个实验输出一个中间过程，实现一个c/c++代码，需要makefile完成项目管理
- 大约需要如下个核心文件，分别为词法 lexer，语法parser，语义 sementic，代码生成 codec和main来实现comiler的主程序调用

```
TARGET = compiler  # 目标文件可执行程序
CFLAGS = -Wall -Werror -std=gnu99 -g -I -no-pie  # 参数 gccc  -no-pie
OBJS = lexer.o codegen.o parser.o verbose.o main.o
deps := $(OBJS:%.o=.%.o.d)
%.o: %.c
    @printf "  CC\t$@\n"
    @$(CC) -o $@ $(CFLAGS) -c -MMD -MF .$@.d $<
$(TARGET): $(OBJS)
    @printf "  LD\t$@\n"
    @$(CC) $(CFLAGS) -o $@ $^
clean:
    $(RM) $(TARGET) $(TESTS) $(OBJS) $(deps) .cbuild
-include $(deps)
```

二、实验七、八内容

- 词法部分，scanner()或者lexer()没有问题，实验2完成度很高
- 语法，Parser，实现类c的语法，需要实现slr分析表，对于可接受的语句进行规约，实验3，4，5是slr部分
- 语义和代码生成部分，体现在符号表设计
- 最简洁的方式，就是从源代码分析，到词法到四元式输出
- 提供了若干参考代码供大家验收使用，验收过程如原创请说出自己工作量或创新